Case Study: Data-Driven Early Identification of At-Risk Students

Student Performance Analytics and Prediction System (Python mini project)

1. Background

A mid-sized engineering college runs four departments: Electronics, Computer, Mechanical and Civil. Every semester some students fall behind, but the problem is usually noticed only after the examination result, when it is too late to help. Attendance registers, assignment sheets and study-habit surveys exist, yet they are kept in separate files and are never analysed together.

The academic coordinator asks a student analyst to build a small, repeatable analysis that shows which habits actually move marks, points out students who need help early, and gives a rough prediction for new students. The analyst decides to use free Python tools: NumPy, Pandas, Matplotlib, Seaborn, SciPy and scikit-learn.

2. Problem Statement

How can the college use its existing student records to understand what drives marks, flag students at risk of failing before the examination, and estimate the result of a new student, using only freely available Python tools?

3. Data and Preparation

The records arrive as CSV and JSON exports with 53 rows and 9 columns (name, age, department, study hours, attendance, assignment score, family income, internet access and marks). A first inspection shows three problems:

- **Missing values:** 3 students have no age and 3 have no attendance figure.
- **Duplicates:** 3 rows were entered twice, which would give those students extra weight in every average.
- **Awkward structure:** names are stored in a single column, so first and last names cannot be used separately.

The analyst removes the duplicates (53 rows become 50), fills the missing ages and attendance values with the column mean (19.89 years and 72.99 percent), splits the name column, and adds two helpful columns: Result (Pass at 50 marks or above) and Grade (A, B, C or F). Mean filling is simple and keeps all 50 students, but it slightly reduces the natural spread of the data, which should be remembered when reading the results.

4. Solution Implemented

The whole solution is one Python program that runs as a pipeline of eight stages, each answering a different question about the data.

Stage	Technique	Question it answers
1. Explore	NumPy arrays, slicing, boolean masks, truth table	What does the data look like, and when should an early-warning alert fire (two or more of three risk flags)?
2. Load and inspect	Pandas read_csv, read_json, info, describe	How large is the data and what is wrong with it?

Stage	Technique	Question it answers
3. Clean	Nulls, duplicates, mean filling, name split	Is the data fit for analysis?
4. Filter and group	Conditions, GroupBy, .agg()	Who is at risk, and how do departments compare?
5. Visualise	Line, bar, histogram, scatter, heatmap	What patterns are visible?
6. Statistics	Mean, variance, Pearson, Spearman	How strong is the relationship with marks?
7. Predict marks	Linear Regression	How many marks does an extra study hour add?
8. Predict result	One-hot encoding + Logistic Regression	Will a student pass or fail?

5. Findings

5.1 Study hours dominate everything else

The correlation between study hours and marks is 0.937 (Pearson) and 0.943 (Spearman), both with p-values far below 0.001, so the relationship is both strong and statistically convincing. The two measures agreeing shows the link is steady and not driven by a few extreme students. Attendance (0.74) and assignment score (0.78) also relate to marks, but they correlate strongly with study hours too (0.80 and 0.84), so part of their effect is the same effect seen through a different column. Age has almost no relationship with marks (0.04).

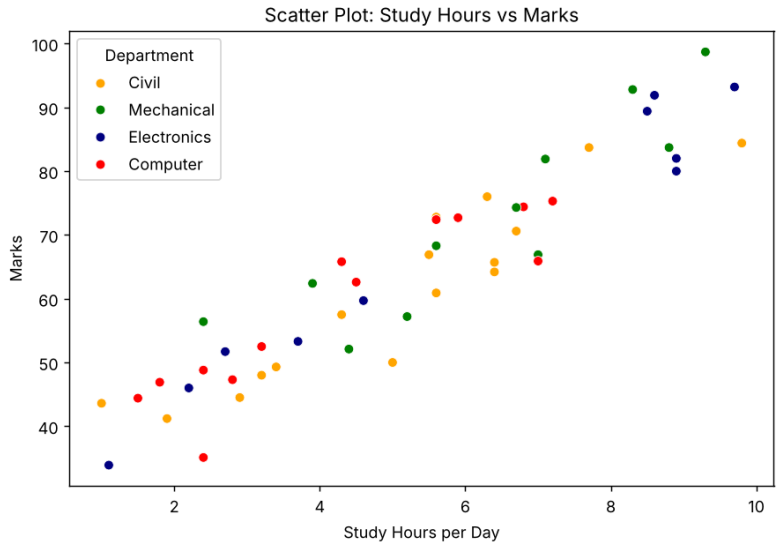


Figure 1: Marks rise steadily with study hours in every department

5.2 Department differences

Department	Average marks	Average attendance	Average study hours
Mechanical	72.25	77.39	6.25
Electronics	68.11	71.98	5.89
Civil	61.21	74.11	5.11
Computer	58.78	68.68	4.26

Mechanical leads and Computer trails. The Computer department also has the lowest study time and attendance, which is consistent with its lower marks. These averages should be read gently: each department has only 12 to 18 students, and splitting them further by family income leaves very small

groups. For example, the Electronics high-income average of 91.30 against 46.30 for low income looks dramatic but rests on very few students and is not proof of anything.

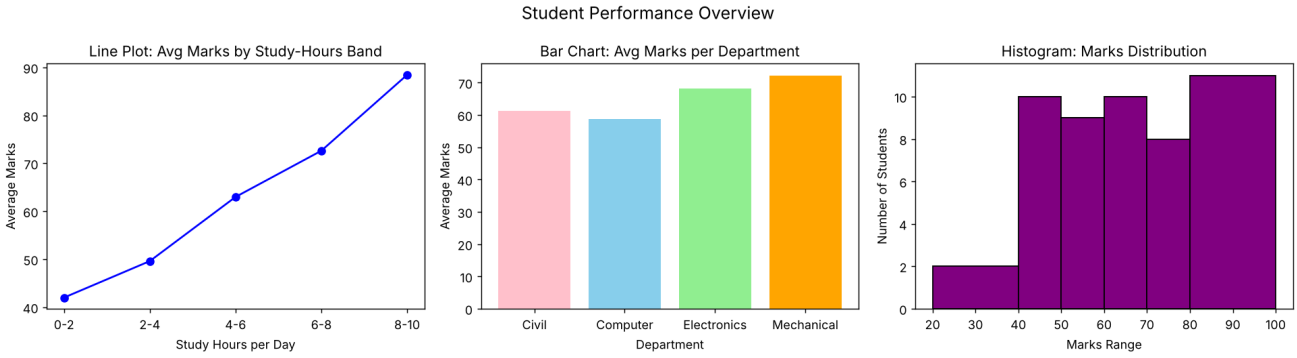


Figure 2: Marks by study-hours band, by department, and the overall distribution

5.3 At-risk students

Sixteen of the 50 students (32 percent) combine attendance below 75 percent with fewer than 4 study hours a day. Most of them score between 33 and 56 marks, so nearly all sit below or just above the pass line. This list takes one line of Pandas to produce and is the most directly useful output of the project, because it turns the analysis into a list of names to contact. Overall, 76 percent of the students pass.

5.4 Prediction models

Model	Key result	Reading
Linear Regression	Slope 6.35, intercept 31.11	Each extra study hour adds about 6 marks; 7.5 hours predicts 78.8 marks
Linear Regression (test set)	MSE 65.15, R2 0.4613	Typical error about 8 marks on 10 test students
Logistic Regression	Accuracy 1.00	13 of 13 test students classified correctly
New student: 3 h/day, 70% attendance	Predicted Fail	Matches the at-risk pattern

Two cautions apply. First, the R2 of 0.46 looks weak next to the correlation of 0.937 because the test set holds only 10 students, so a handful of unusual points can pull the score down; across all 50 students, study hours explain roughly 88 percent of the variation in marks. Second, the perfect classification score comes from a test set of only 13 students, of whom 3 failed. It is encouraging but not strong evidence, and the model would need more data and cross-validation before anyone relies on it.

6. Recommendations

- **Target the at-risk students early.** Run the filter every month and offer mentoring, extra practice sessions or study groups to those flagged.
- **Focus on study routine, not only attendance.** Marks respond most to study hours, so study-planning workshops are likely to help more than attendance reminders alone.
- **Support the Computer department first.** It shows the lowest study time, attendance and marks, so it has the most room to improve.
- **Keep records clean at the source.** Duplicates and blanks appeared in only 53 rows; in a full college they would affect hundreds. Simple form validation saves repeated cleaning.
- **Grow the data before trusting predictions.** Use several semesters of real records, cross-validation and more features before using the models for decisions about individual students.

7. Limitations

- The dataset is synthetic and was built so that study hours influence marks, so the strong correlation is partly by design. Real data will be noisier.
- Correlation does not prove cause. Motivated students may both study more and score more.
- Mean filling of missing values is basic. Median filling, grouped means or model-based filling may suit real data better.
- Small samples make department and income comparisons and model scores unstable.
- Predictions about people should support teachers' judgement, never replace it.

8. Conclusion

The case shows that NumPy, Pandas, Matplotlib, Seaborn, SciPy and scikit-learn fit together into a complete analytics workflow: collect, clean, explore, visualise, measure and predict. With one short Python program the college can move from raw registers to a monthly list of students who need help, a clear message that study habits matter most, and a first prediction tool. The main lesson is that each technique answers a different question, and the value comes from using them in sequence while staying honest about how much a small dataset can prove.